PathPilot: Interactive Pathfinding Learning Platform

This is Submitted By:

Mudit Somra

Section-9S394

Under the Guidance of
Prerita Agarwal (68535)

School of Computing & Artificial Intelligence
Lovely Professional University, Phagwara

DECLARATION

I hereby certify that the project titled “PathPilot – Interactive Pathfinding Learning Platform” is a genuine representation of my work, conducted as part of the Project requirement for the course CSE001 at Lovely Professional University, Phagwara. This work was carried out under the supervision of Professor Prerita Agarwal during the period of June to August 2026. All the work, analysis, and findings presented in this project report are based on my own efforts and represent my original contribution.

Name of Student : Mudit Somra
Registration No : 12514098

ABSTRACT

The increasing importance of Data Structures and Algorithms (DSA) in software development and technical interviews has created a need for interactive learning tools that simplify complex algorithmic concepts. During the Summer Preparation Course conducted from June to August 2026 under the guidance of Ms. Prerita Agarwal at Lovely Professional University, various DSA topics such as arrays, searching, sorting, hashing, linked lists, stacks, queues, trees, graphs, greedy algorithms, recursion, backtracking, and dynamic programming were studied through problem-solving and competitive programming practice.

Inspired by the graph algorithms covered during the course, this project presents PathPilot – An Interactive Pathfinding Learning Platform, a web-based application developed using React, Vite, Tailwind CSS, and JavaScript. The application enables users to visualize popular pathfinding algorithms, including Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search, on an interactive grid where users can place walls, reposition source and destination nodes, and generate random mazes. Animated visualization of node exploration and shortest-path construction helps learners understand the execution flow of each algorithm.

To further enhance the learning experience, the project incorporates a dedicated Learning Mode that provides algorithm theory, time and space complexity analysis, advantages, limitations, and practical applications. During algorithm execution, the Learning Mode synchronizes with the visualization to display execution timelines, current step explanations, current node information, live visualization of internal data structures such as queues, and execution statistics. These features enable users to understand both the graphical progression and the internal working of each algorithm. Additionally, the modular architecture has been designed to support future enhancements such as synchronized pseudocode highlighting, replay functionality, weighted graph visualization, and the integration of additional algorithms including Dijkstra's Algorithm, Greedy Best-First Search, and Bidirectional Breadth-First Search.

The primary objective of PathPilot is not only to demonstrate algorithm execution but also to improve conceptual understanding through interactive visualization and structured educational content. The project serves as an effective learning platform for students preparing for competitive programming, coding interviews, and graph algorithm fundamentals while demonstrating modern frontend development practices and clean software architecture.

TABLE OF CONTENT

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

The understanding of graph traversal and pathfinding algorithms is a fundamental requirement for students preparing for competitive programming, software engineering interviews, and computer science courses. Although numerous online resources explain algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search, most learning platforms either focus only on theoretical concepts or provide simple visual demonstrations without explaining the internal execution of these algorithms. As a result, many students memorize algorithmic steps instead of developing a clear conceptual understanding of how the algorithms explore nodes, manage their internal data structures, and determine the shortest path. [1][2]

Existing pathfinding visualizers generally display only the final animation of visited nodes and the computed shortest path. However, they rarely provide detailed explanations of algorithm decisions, live visualization of supporting data structures such as queues, stacks, or priority queues, or synchronized pseudocode execution. This makes it difficult for beginners to relate the theoretical concepts learned in textbooks with the actual execution process of an algorithm. [1]

To address these limitations, the proposed project, PathPilot – Interactive Pathfinding Learning Platform, aims to provide an integrated educational environment where users can not only visualize pathfinding algorithms but also understand their internal working through interactive learning modules. The platform combines algorithm visualization with theoretical explanations, complexity analysis, maze generation, and a dedicated Learning Mode. Future enhancements include pseudocode highlighting, replay functionality, and step-by-step explanations, enabling learners to observe every stage of algorithm execution in an intuitive manner. By combining visualization with educational content, PathPilot seeks to improve conceptual understanding, problem-solving ability, and practical knowledge of graph algorithms.

1.2 Motivation

The motivation behind developing PathPilot – Interactive Pathfinding Learning Platform originated from the challenges encountered while studying graph algorithms during the Summer Preparation Course at Lovely Professional University. Throughout the course, students practiced numerous Data Structures and Algorithms (DSA) problems to strengthen their competitive programming and interview preparation skills. Although theoretical explanations and coding exercises helped in understanding algorithm implementation,

visualizing how graph traversal algorithms explored nodes and reached the destination remained difficult for many learners.

Traditional learning resources such as textbooks and online tutorials generally explain the logic of algorithms using static diagrams or pseudocode. While these resources are valuable, they often fail to demonstrate how algorithms behave dynamically during execution. Existing online pathfinding visualizers effectively display the movement of algorithms across a grid but usually provide limited educational support regarding why a particular node is selected, how internal data structures evolve, and how algorithmic decisions are made during execution. This gap between theoretical knowledge and practical visualization inspired the development of PathPilot. [1][2]

The primary motivation of this project is to create an interactive learning platform that combines visualization with conceptual understanding. Rather than only displaying the final shortest path, PathPilot aims to explain each stage of algorithm execution in an intuitive and engaging manner. The application enables users to construct custom grids, place obstacles, reposition source and destination nodes, generate random mazes, and visualize multiple pathfinding algorithms through smooth animations.

Another important motivation is to encourage self-paced learning. The Learning Mode extends the application beyond visualization by integrating algorithm theory, complexity analysis, execution timelines, live data structure visualization, current node information, execution statistics, and step-by-step explanations of algorithm execution. Future enhancements include synchronized pseudocode highlighting, replay functionality, and support for additional pathfinding algorithms. These features are intended to help learners understand not only what an algorithm does but also why it behaves in a particular manner during execution.

From a software engineering perspective, the project also provided an opportunity to apply modern frontend development practices using React, Vite, and Tailwind CSS while following a modular architecture. By separating the visualization engine, grid management, algorithms, animation logic, and learning components, the application has been designed to support future enhancements with minimal modifications, making it scalable and maintainable. [3]

1.3 Objectives of the Study

The primary objectives of the proposed project are as follows:

• To develop an interactive web-based pathfinding visualization platform.
The project aims to design and develop an intuitive web application that enables users to visualize the execution of pathfinding algorithms on a two-dimensional grid. The platform

provides an interactive environment where users can experiment with different scenarios and better understand graph traversal techniques.

• To implement multiple pathfinding algorithms for comparative learning.
The application incorporates Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms. Each algorithm demonstrates a unique traversal strategy, allowing users to compare their behavior, efficiency, and suitability for different pathfinding problems.

• To provide an interactive grid environment for experimentation.
The project enables users to create and remove walls, reposition the start and destination nodes, generate random mazes, and execute algorithms on customized grid layouts. These interactive features allow users to observe how changes in the environment influence algorithm performance.

• To integrate an educational Learning Mode within the application.
In addition to visualization, the project includes a dedicated Learning Mode that presents theoretical information related to the selected algorithm. This includes its working principle, computational complexity, advantages, disadvantages, and practical applications, making the platform useful for both learning and revision.

• To design a modular and scalable software architecture.
The application is developed using a modular architecture that separates the user interface, grid management, animation engine, and algorithm implementations. This approach improves maintainability and allows new algorithms and educational features to be incorporated with minimal modifications to the existing system.

• To enhance the understanding of graph algorithms through visualization.
The project aims to simplify the learning process by combining algorithm execution with graphical animation and interactive features. This helps students, educators, and programming enthusiasts develop a deeper understanding of pathfinding algorithms while supporting academic learning, technical interview preparation, and competitive programming.

CHAPTER 2
LITERATURE REVIEW

2.1 Related Work

Graph algorithms play a significant role in solving real-world problems involving shortest path computation, route optimization, network communication, robotics, and artificial intelligence. Because these algorithms involve multiple intermediate steps and dynamic decision-making, understanding their execution through traditional textbooks alone can be challenging. Consequently, several visualization tools and educational platforms have been developed to simplify the learning process by providing graphical demonstrations of algorithm behavior. [1][2]

One of the most popular pathfinding visualization tools is Pathfinding Visualizer, developed by Clement Mihailescu. The platform allows users to create custom grids, place obstacles, generate random mazes, and visualize the execution of various pathfinding algorithms such as Breadth-First Search (BFS), Dijkstra's Algorithm, Greedy Best-First Search, and A* Search. The application provides smooth animations that clearly illustrate how nodes are explored before the shortest path is identified. Due to its intuitive interface and interactive design, the platform has become widely used by students preparing for coding interviews and competitive programming. However, the primary focus of the application is visualization, with relatively limited emphasis on explaining the underlying theory, algorithmic decisions, or internal data structures involved during execution. [4]

Another widely recognized educational platform is VisuAlgo, which provides interactive demonstrations of numerous data structures and algorithms, including graph traversal techniques. In addition to animations, VisuAlgo presents pseudocode, complexity analysis, and textual explanations to help learners understand algorithm behavior. Although the platform is highly effective for teaching general algorithmic concepts, it is not specifically designed as a pathfinding simulator. Users cannot freely construct grid environments, generate custom mazes, or interactively modify source and destination nodes while observing algorithm execution. [5]

Apart from dedicated visualization platforms, numerous educational websites and programming resources explain graph algorithms using static diagrams, flowcharts, and code examples. While these resources provide valuable theoretical knowledge, they often lack interactive execution, making it difficult for beginners to understand how algorithms behave in dynamic environments. Students frequently need to switch between multiple resources, such as textbooks, documentation, and visualization tools, to fully grasp both the theoretical and practical aspects of graph traversal algorithms. [1][2]

The development of modern frontend technologies has significantly improved the creation of interactive educational applications. React provides a component-based architecture that enables reusable user interface components and efficient state management, making it particularly suitable for applications requiring continuous interface updates during algorithm execution. Vite offers a lightweight and fast development environment with optimized production builds, while Tailwind CSS simplifies responsive user interface development through utility-first styling. Together, these technologies enable the development of scalable and maintainable web applications with improved performance and user experience. [3][6][7]

Although the existing platforms provide valuable learning resources, several limitations remain. Most applications concentrate primarily on visualizing the traversal process and the final shortest path without adequately explaining why specific nodes are selected, how internal data structures evolve, or how algorithmic decisions are made during execution. Furthermore, educational content is often presented separately from visualization, requiring learners to divide their attention between different resources.

To address these limitations, PathPilot integrates algorithm visualization with educational content in a single application. The platform supports interactive grid creation, wall placement, node movement, maze generation, and a dedicated Learning Mode that presents algorithm theory, computational complexity, execution timelines, current step explanations, current node information, live visualization of algorithm-specific data structures, and execution statistics. The modular architecture also supports future enhancements such as synchronized pseudocode highlighting, replay functionality, weighted graph visualization, and additional pathfinding algorithms

CHAPTER 3
METHODOLOGY

3.1 System Overview

PathPilot was developed using a modular software architecture to ensure scalability, maintainability, and ease of future enhancements. The application follows a component-based approach in which the user interface, pathfinding algorithms, animation engine, grid management, and educational features are implemented as independent modules. This separation of concerns improves code readability and allows individual components to be modified without affecting the overall system.

The application begins by generating an interactive two-dimensional grid consisting of rows and columns. Users can place walls, reposition the source and destination nodes, generate random mazes, and select a pathfinding algorithm for visualization. Once the visualization starts, the selected algorithm computes the traversal path while the animation engine progressively updates the grid to display visited nodes and the final shortest path. The integrated Learning Mode provides theoretical information related to the selected algorithm and has been designed to support future educational features such as execution statistics, live data structure visualization, and step-by-step explanations.

3.2 System Architecture

The application follows a layered architecture in which each module performs a specific responsibility.

- Presentation Layer: Developed using React components to provide the user interface.
- Grid Management Layer: Responsible for creating and updating the grid, handling user interactions, and managing cell states.
- Algorithm Layer: Contains independent implementations of BFS, DFS, and A* Search.
- Animation Layer: Controls the visualization speed and sequential rendering of visited nodes and shortest paths.
- Learning Module: Displays algorithm theory and is designed to support additional educational features.

This modular design minimizes code duplication and simplifies future integration of additional pathfinding algorithms and learning tools.

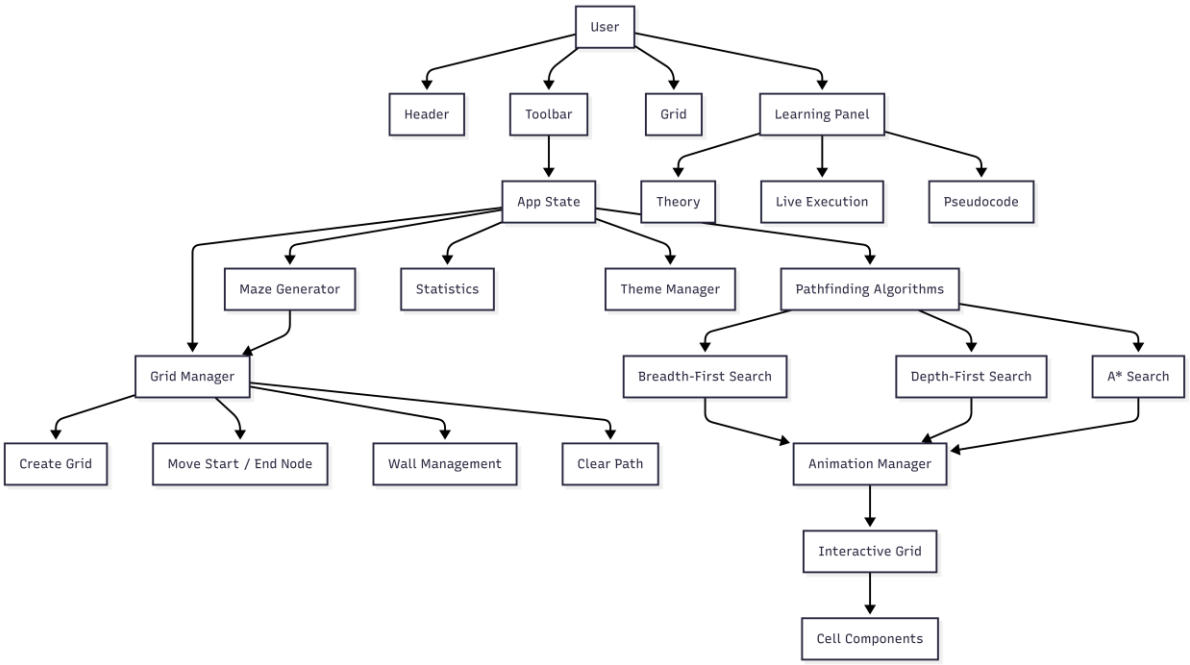

Fig 1: System Architecture

3.3 Technology Stack

The project was developed using modern web development technologies.

Technology	Purpose
React	Component-based frontend development
Vite	Fast development server and build tool
JavaScript (ES6+)	Application logic
Tailwind CSS	User interface styling
HTML5	Page structure
CSS3	Additional styling
Git & GitHub	Version control

React was selected because of its reusable component architecture and efficient state management, while Vite provides faster development and optimized production builds.

Tailwind CSS was used to develop a responsive and consistent user interface with minimal custom CSS. **[3][6][7]**

3.4 Grid Generation

The grid serves as the foundation of the PathPilot application. It provides an interactive environment on which pathfinding algorithms operate. During application initialization, a two-dimensional array is generated where each element represents an individual cell of the grid. Every cell maintains information about its position and current state, including whether it is an empty cell, wall, start node, end node, visited node, or part of the shortest path.

Each cell is represented as an object containing properties required by the visualization engine. These properties allow the application to update only the affected cells during animation while maintaining the overall structure of the grid. React state management is used to store the grid, ensuring that any modification, such as creating walls or moving nodes, automatically updates the user interface.

The default grid dimensions are fixed during initialization to provide a consistent visualization area. Users can interact with the grid by dragging the start and destination nodes, drawing or removing walls, and generating random mazes before executing a pathfinding algorithm.

The modular design of the grid management system separates grid creation, state updates, and rendering logic, making future enhancements easier to implement without affecting other components of the application.

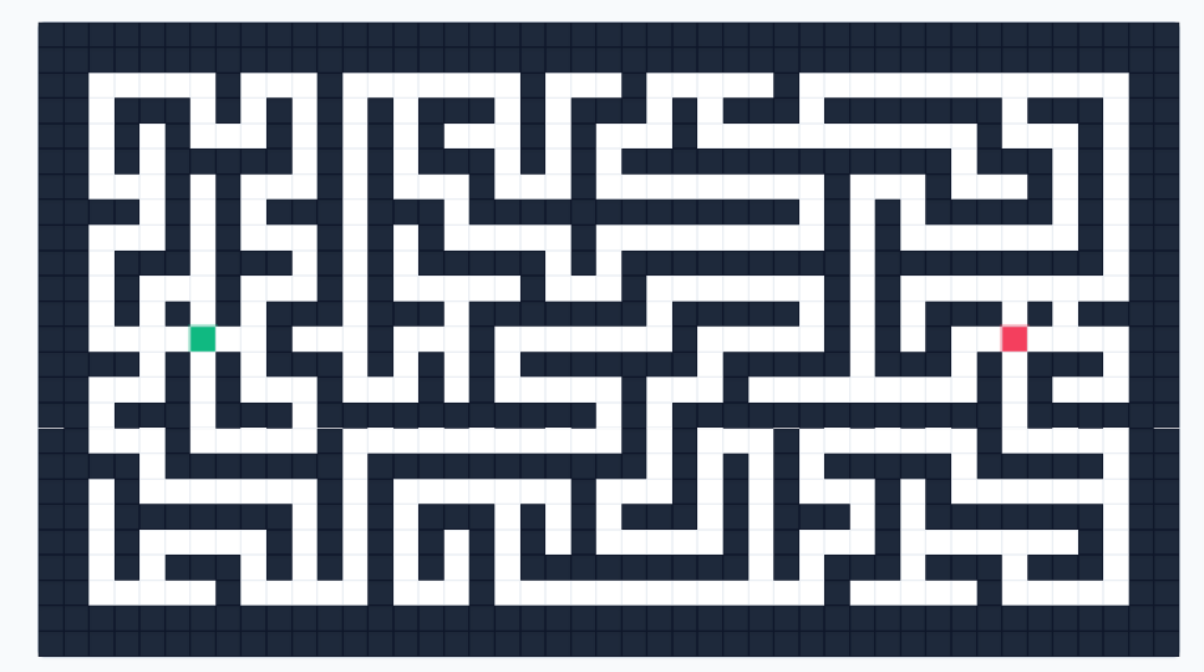

Fig 2: Interactive Grid

3.5 Pathfinding Algorithms

The application currently supports three pathfinding algorithms: Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search. Each algorithm is implemented as an independent module, allowing new algorithms to be added without modifying the visualization engine.

Before execution begins, the application identifies the start node and destination node from the current grid configuration. The selected algorithm processes the grid while avoiding blocked cells and records the order in which nodes are visited. After execution, the visualization engine animates the visited nodes followed by the shortest path if one exists.

Breadth-First Search (BFS)

Breadth-First Search explores neighbouring nodes level by level using a queue data structure. Since nodes are explored in increasing order of their distance from the source, BFS guarantees the shortest path in an unweighted graph. [1]

Depth-First Search (DFS)

Depth-First Search explores one path completely before backtracking to previously visited nodes. It uses a stack (or recursion) to manage traversal order. Although DFS can successfully locate the destination, it does not always produce the shortest path. [1]

A* Search

A* Search combines the actual distance travelled with a heuristic estimate of the remaining distance to efficiently determine the optimal path. By prioritizing nodes with the lowest estimated total cost, A* generally explores fewer nodes than BFS while still guaranteeing the shortest path when an admissible heuristic is used. [8]

Fig 3: Visualization of BFS Algorithm

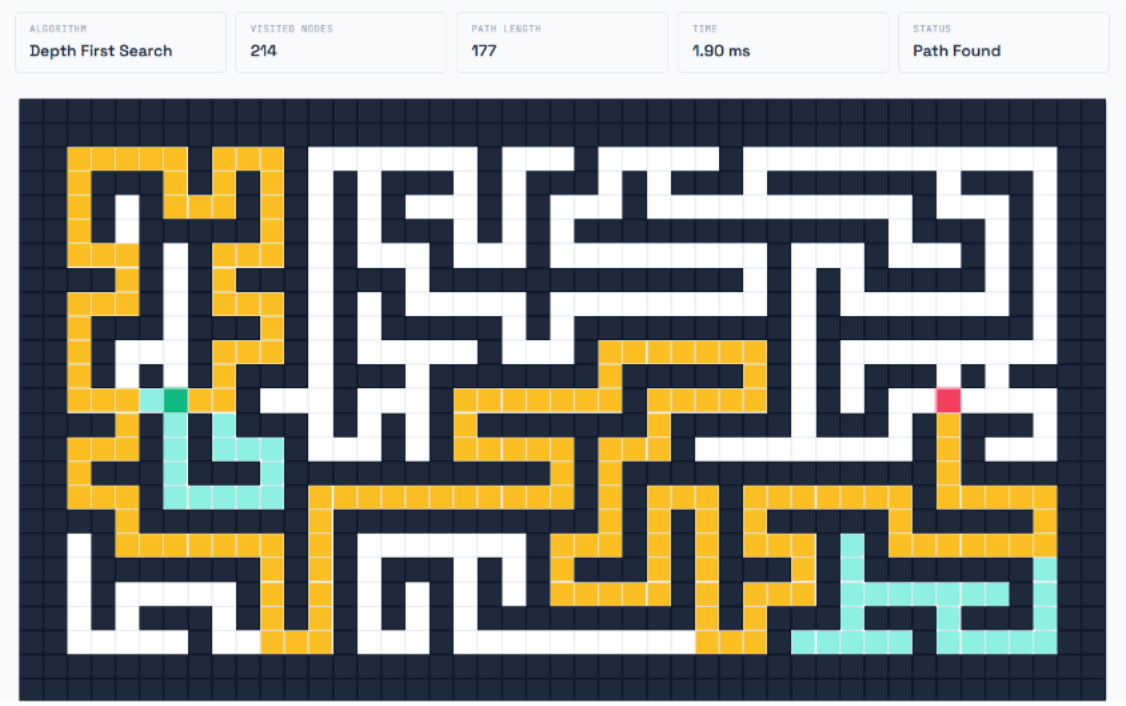

Fig 4: Visualization of DFS Algorithm

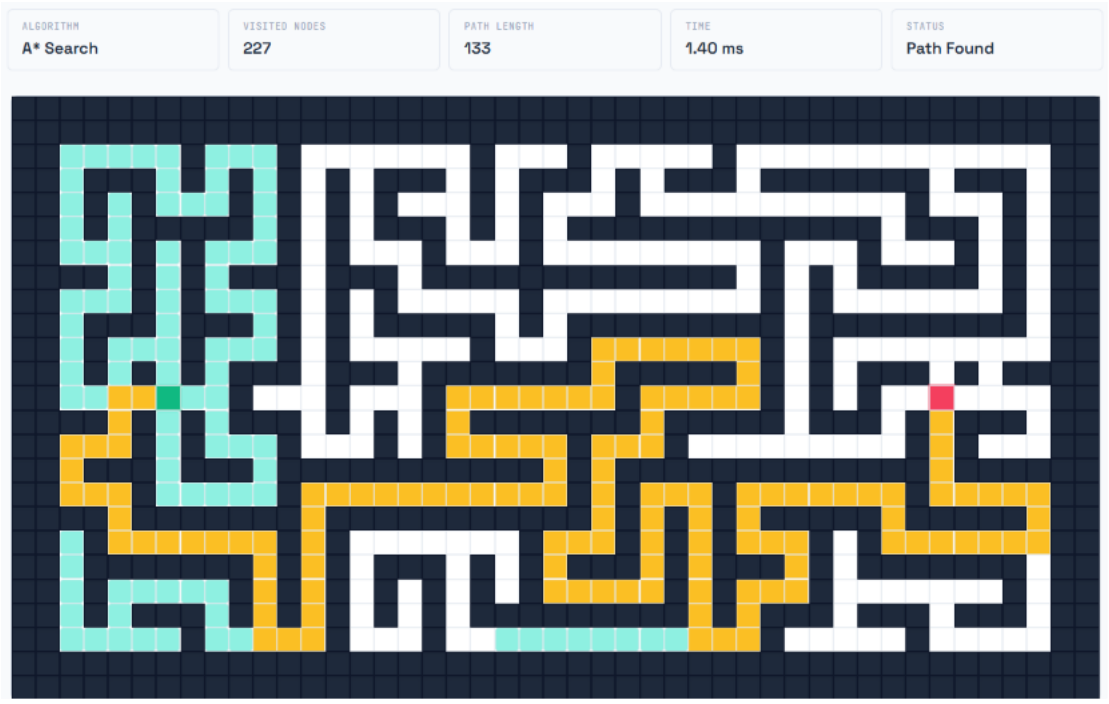

Fig 5: Visualization of A* Algorithm

3.6 Animation Mechanism

The animation mechanism is responsible for converting the output of the selected pathfinding algorithm into a smooth and interactive visualization. Instead of directly updating the entire grid after an algorithm completes, the application animates each step sequentially to help users observe how the algorithm explores nodes and eventually discovers the shortest path.

When a pathfinding algorithm finishes execution, it returns two collections of data. The first contains the order in which nodes were visited during traversal, while the second contains the nodes that form the shortest path between the source and destination. These collections are passed to the animation manager, which updates the grid one node at a time.

The visualization process is divided into two stages. During the first stage, all visited nodes are animated sequentially to demonstrate the exploration pattern of the selected algorithm. Once all visited nodes have been displayed, the second stage highlights the nodes that form the final shortest path. This separation helps users clearly distinguish between the search process and the final solution.

The animation speed can be adjusted by the user through the toolbar. Different speed settings allow users to either carefully observe each execution step or complete the visualization quickly. During animation, user interactions such as wall creation, maze generation, and node movement are temporarily disabled to maintain consistency and prevent unintended modifications to the grid.

The animation module operates independently of the pathfinding algorithms, enabling different algorithms to reuse the same visualization engine. This modular design simplifies maintenance and allows future algorithms to be integrated without requiring changes to the animation logic.

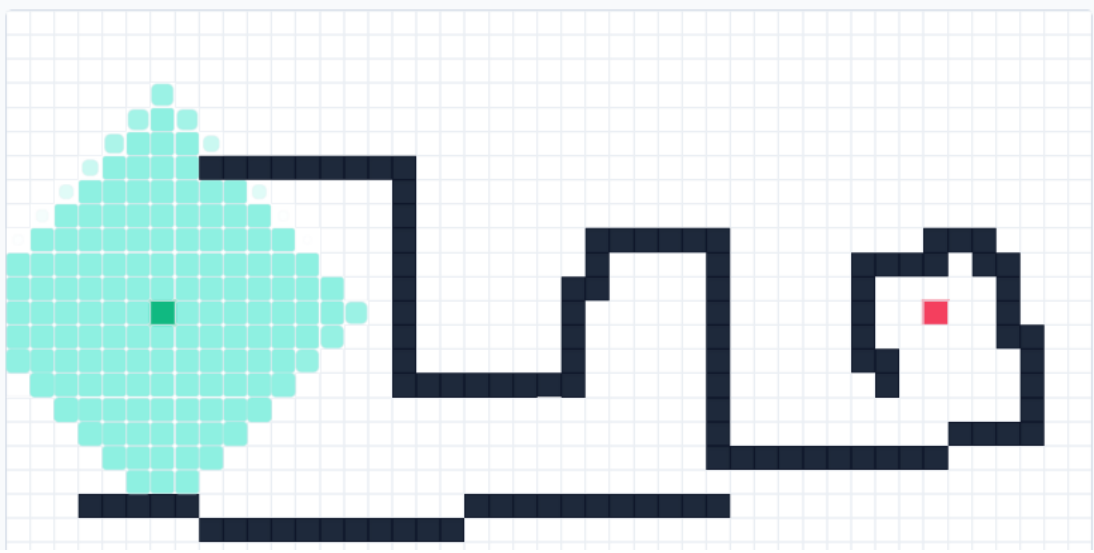

Fig 6: Node Exploration during Algorithm execution

3.7 Learning Mode

The Learning Mode extends the application beyond traditional algorithm visualization by providing educational content alongside the interactive grid. Instead of only displaying the execution of an algorithm, this mode is designed to help users understand the underlying concepts through theoretical explanations and algorithm-specific information.

Users can switch between the standard visualization mode and Learning Mode using the application toolbar. When Learning Mode is enabled, a dedicated side panel is displayed containing information related to the selected algorithm. The current implementation includes sections describing the algorithm overview, working principle, time complexity, space complexity, advantages, disadvantages, and practical applications.

The Learning Mode has been designed using a modular architecture to support future enhancements without modifying the existing visualization engine. The current implementation of Learning Mode includes execution timelines, current step explanations, current node information, live visualization of algorithm-specific data structures, and execution statistics synchronized with the animation. These features enable users to observe not only the graphical visualization but also the internal execution process of the selected algorithm. The modular architecture also supports future enhancements such as synchronized pseudocode highlighting, replay functionality, weighted graph visualization, and the addition of new pathfinding algorithms.

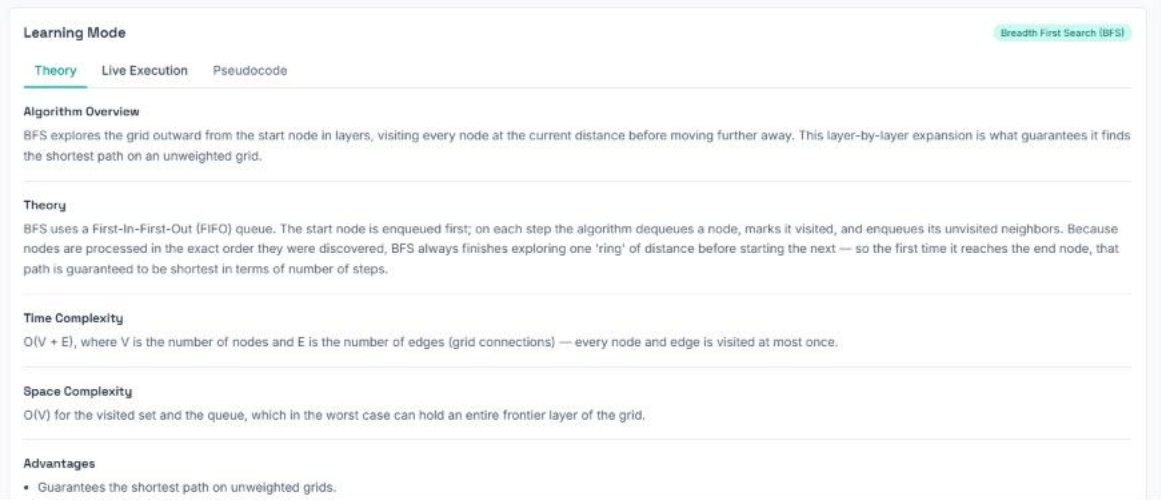

Fig 7: Learning Mode Interface

3.8 Summary of Methodology

The methodology adopted for the development of PathPilot emphasizes modularity, scalability, and ease of maintenance. Independent implementation of the grid management system, visualization engine, pathfinding algorithms, and educational components allows each module to function separately while contributing to the overall operation of the application. This architecture simplifies future development and supports the addition of new algorithms and learning features with minimal changes to the existing codebase.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Experimental discussion

The developed PathPilot application was evaluated by executing different pathfinding algorithms on multiple grid configurations, including empty grids, custom obstacle layouts, and randomly generated mazes. The evaluation focused on the correctness of pathfinding, visualization behavior, user interaction, and overall learning experience rather than predictive accuracy.

During experimentation, Breadth-First Search (BFS) consistently produced the shortest path in all unweighted grid configurations. Depth-First Search (DFS) successfully reached the destination whenever a valid path existed but did not always generate the shortest route because it explores one branch completely before backtracking. A* Search demonstrated more efficient exploration by visiting fewer nodes while still producing the optimal path through heuristic guidance.

The maze generation module successfully created different obstacle layouts for repeated experimentation, allowing users to observe algorithm behavior under varying levels of complexity. The animation engine displayed visited nodes sequentially before highlighting the final shortest path, making the execution process easier to understand.

4.2 Testing

Functional testing confirmed that all core features operated as expected. Users were able to modify the grid, execute pathfinding algorithms, generate random mazes, and switch between visualization and Learning Mode without affecting application stability.

Test Case	Expected Result	Status
Grid Initialization	20 × 40 grid generated	Pass
Draw Walls	Walls created correctly	Pass
Move Start Node	Node relocated successfully	Pass
Move End Node	Node relocated successfully	Pass
BFS Visualization	Shortest path displayed	Pass
DFS Visualization	Valid path displayed	Pass
A* Visualization	Optimal path displayed	Pass

Test Case	Expected Result	Status
Maze Generation	Random maze created	Pass
Learning Mode	Theory displayed correctly	Pass
Reset Grid	Grid restored	Pass

4.3 Performance Insights

Experimental observations indicate that BFS explores nodes uniformly across the grid, whereas DFS follows a depth-oriented exploration strategy. A* Search generally visits fewer nodes because the heuristic function guides exploration toward the destination. As a result, A* provides a faster and more efficient visualization in most scenarios while maintaining the optimal path.

Feature	BFS	DFS	A*
Finds Path	Yes	Yes	Yes
Guarantees Shortest Path	Yes	No	Yes
Main Data Structure	Queue	Stack	Priority Queue
Suitable for Unweighted Graphs	Yes	Moderate	Yes
Search Efficiency	Moderate	Moderate	High

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The PathPilot project successfully achieved its primary objective of developing an interactive platform for visualizing and understanding pathfinding algorithms. The application combines algorithm visualization with an intuitive user interface, allowing users to create custom grid environments, place obstacles, reposition source and destination nodes, generate random mazes, and visualize the execution of Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms.

Unlike conventional pathfinding visualizers that focus primarily on animation, PathPilot incorporates an educational perspective through the introduction of a dedicated Learning Mode. This feature provides theoretical information about the selected algorithm, including its working principle, computational complexity, advantages, disadvantages, and practical applications. The modular architecture adopted during development separates the user interface, grid management, animation engine, maze generation, and algorithm implementations, resulting in a scalable and maintainable system.

The project also provided practical experience in modern frontend development using React, Vite, and Tailwind CSS while reinforcing concepts related to graph algorithms, software design, and component-based application development. Overall, the developed application serves as both an educational resource and a practical demonstration of algorithm visualization techniques.

5.2 Future Work

Although the current version of PathPilot provides a complete visualization platform for BFS, DFS, and A* Search, several enhancements can further improve its educational value and functionality. Future versions of the application may include:

- Integration of additional pathfinding algorithms such as Dijkstra's Algorithm, Greedy Best-First Search, Bidirectional Breadth-First Search, and Bellman-Ford Algorithm.
- Synchronized pseudocode highlighting to display the currently executing line during algorithm visualization.
- A replay and step-by-step execution system that allows users to pause, resume, rewind, and navigate through individual stages of algorithm execution.
- Support for weighted grids and weighted pathfinding algorithms to simulate more realistic navigation scenarios.
- Interactive quizzes and coding challenges to reinforce theoretical concepts and evaluate users' understanding of graph algorithms.
- Additional educational modules covering advanced graph algorithms and real-world pathfinding applications.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[2] S. S. Skiena, The Algorithm Design Manual, 3rd ed., Springer, 2020.

[3] React Documentation. Available: https://react.dev/ (Accessed: August 2026).

[4] C. Mihailescu, *Pathfinding Visualizer*. Available: https://clementmihailescu.github.io/Pathfinding-Visualizer/ (Accessed: August 2026).

[5] VisuAlgo, *Visualising Data Structures and Algorithms*. Available: https://visualgo.net/ (Accessed: August 2026).

[6] Vite Documentation. Available: https://vite.dev/ (Accessed: August 2026).

[7] Tailwind CSS Documentation. Available: https://tailwindcss.com/ (Accessed: August 2026).

[8] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.